Software python

Installing Python Packages

Python is provided on Engaging through a set of `miniforge` modules. Check `module avail miniforge` to see a list of available versions. Miniforge is a conda distribution created by the community that maintains the conda-forge channel, is open source, and doesn't require purchasing a license to use.

To load the latest version of `miniforge` you can run:

```
module load miniforge
```

or specify a version with:

```
module load miniforge/25.11.0-0
```

Python packages will need to be installed in your home directory or other directory you have write access to. There are generally two ways to do this. At a high level, you can:

- Install packages in your own Python virtual environment (venv)
- Install packages in your own conda/mamba environment

Which you use is mostly personal preference, but sometimes it could depend on which packages your are installing. We encourage everyone to use environments, which can be created per project and will have fewer issues with package compatibility. Read through the pros and cons for each, they are meant to help you see when one might be better than another.

Environments allow you to make self-contained “bundles” of packages that can be loaded and unloaded. This helps keep a consistent set of packages and versions for a given project, rather than putting all packages you've ever installed together.

Python Virtual Environments

Python virtual environments are a native python functionality and can be placed anywhere you have write access to and have all their packages in that environment’s directory structure.

Creating Python Virtual Environments

To create a new environment, first load the `miniforge` module using the `module load` command. See the page on Modules for more information on how to load modules.

```
module load miniforge/25.11.0-0
```

To create a the environment use the `python -m venv` command:

```
python -m venv /path/to/virtual/environment
```

Note that you specify a path, and not only a name for the environment. This means you can place your environment wherever you'd like. Usually it is placed somewhere in the project directory it is used for. For example, if I want to create an environment for a project called `my_project` that lives in my home directory, I would run something like:

```
cd ~/my_project
python3 -m venv my_project_env
```

This will create an environment at the path `~/my_project/my_project_env`.

Note

Virtual Environment documentation and tutorials will often tell you to run the command `python3 -m venv .venv` to create an environment. The `.` at the beginning of `.venv` creates a hidden directory in the current working directory. You won't see this directory if you run the `ls` command by itself, run `ls -a` to see hidden files and directories. If you are new to virtual environments and Linux we recommend using a descriptive name for your environment, such as `projectname_venv`.

By default environments will be fully isolated and Python will only see the packages you've installed in the environment. You can signal your virtual environment to "see" system installed packages by using the flag `--system-site-packages` when creating the environment. This is useful when the miniforge module has some of the packages you are using. For example:

```
python3 -m venv --system-site-packages my_project_env
```

Before installing packages or using an environment we need to activate the environment:

```
source /path/to/virtual/environment/bin/activate
```

Using our example above, if we are in the `my_project` directory already, we can run:

```
source my_project_env/bin/activate
```

We can now install packages into the environment. Once activating the environment new packages can be installed with the `pip` command:

```
pip install pkgname
```

> **Do not use the --user flag**
>
> Do not use `--user` flag to install packages with `pip`, this will install the package into `$HOME/.local` instead of the environment.

When you are done using or installing packages into an environment you can deactivate it with the command:

```
deactivate
```

Using Python Virtual Environments

In order to use an environment you will need to activate it as described above:

```
source /path/to/virtual/environment/bin/activate
```

Using our example above, if we are in the `my_project` directory already, we can run:

```
source my_project_env/bin/activate
```

Once it is activated any `python` commands will run in that environment and have access to the packages in the environment.

To use an environment in a job include the command above in your job script. An environment activated on the login node will not necessarily carry over to your job. Using the same example above, let's look at the script `myjob.sh` in the `my_project` directory:

myjob.sh

```
#!/bin/bash
```

```
# Activate the environment
source my_project_env/bin/activate

# Run the python script
python myscript.py
```

When this job runs it will activate the environment in `my_project_env` and then run the python script `myscript.py` using the packages in that environment.

Exporting and Recreating and Virtual Environments

Environments can be described by a `requirements.txt` file, which lists packages and optionally their versions. This file can be created from any existing virtual environment and used to re-create that environment. Version numbers are required to recreate the environment exactly.

You can either create a `requirements.txt` file by hand by creating a file with one package name on each line, or you can create one from a currently active environment with the command:

```
pip freeze > requirements.txt
```

Given a `requirements.txt` file you can use `pip` to install the packages in that file into your environment. First activate the environment, then install the packages with:

```
source /path/to/virtual/environment/bin/activate
pip install -r requirements.txt
```

Pros and Cons for Virtual Environments

Pros

- Can be set to build “on top of” the central installation packages by using the `--system-site-packages` flag when creating the environment.
- Self-contained environments for each project help stay organized and avoid package dependency conflicts. For software development it allows you to keep better track of your package’s dependencies so others know what they need to install.
- Virtual environments are self-contained, fairly lightweight and can be put anywhere easily without additional configuration files.

Cons

- Environments will include the same version of the python that you used to create them. If you need a different version of Python, you have to create the environment with that other version.

- Only installs packages available through PyPI, cannot install conda distributed packages or libraries.

Conda Environments

Conda environments are a bit different from Python virtual environments, by default they are stored in the `.conda` directory in your home directory. They also tend to contain a lot more files than Virtual Environments. You can also install some system libraries into conda environments, which can make installing packages with system library dependencies easier.

You'll also see mention of mamba environments. Mamba and conda are nearly the same, however mamba has a different dependency solver. Mamba is often better and faster than conda at solving dependencies and picking packages, so we recommend using mamba whenever possible.

Creating Conda Environments

To create a new environment, first load the `miniforge` module using the `module load` command. See the page on Modules for more information on how to load modules.

> ⚠ **Avoid installing your own conda**
>
> We discourage installing miniconda, miniforge, or anaconda in your home directory. Engaging keeps up to date miniforge modules so installing your own is not necessary and will take up space in your home directory and potentially slow down your applications.

To create an environment you can use the `mamba create` or `conda create` command after loading the `miniforge` module:

```
module load miniforge
mamba create -n my_env python=3.12 pkg1 pkg2 pkg3
```

or, using `conda`:

```
module load miniforge
conda create -n my_env python=3.12 pkg1 pkg2 pkg3
```

where `miniforge` loads the latest miniforge module. In this example I am creating a conda environment named `my_env` with Python 3.12 and installing packages pkg1, pkg2, pkg3.

You can install additional packages by activating the environment and using the `mamba install` or `conda install` command. Again you would first load the `miniforge` module if it isn't already loaded:

```
module load miniforge
source activate my_env
mamba install pkg4
```

where `miniforge` loads the latest miniforge module. Here I am installing with `mamba`. Packages that aren't available through conda channels can be installed with the `pip` command when the environment is activated:

```
module load miniforge
source activate my_env
pip install pkg5
```

> **Do not use the --user flag**
>
> Do not use `--user` flag to install packages with `pip`, this will install the package into `$HOME/.local` instead of the environment.

If you would like to use your conda environment in Jupyter, install the `jupyter` package into your environment. Once you have done that, you should see your conda environment listed in the available kernels.

Using Conda Environments

Whenever you want to activate the environment, first load the miniforge module, then activate with `source activate my_env`. The `conda activate` or `mamba activate` commands should also work as long as you load the miniforge module.

> **Do not run conda/mamba init**
>
> Do not run `conda init bash` or `mamba init bash`, this makes changes to your environment that can affect how other applications behave, and sometimes cause errors. Instead run `module load miniforge` before activating your environment. See Conda/Mamba Init in the Troubleshooting section below for more information.

```
module load miniforge
```

```
source activate my_env
```

To use a conda environment in a job you can add these lines to you job script:

myjob.sh

```
#!/bin/bash

# Load the miniforge module
module load miniforge

# Activate your environment
source activate my_env

# Run your Python script
python myscript.py
```

If this isn't working, see the Troubleshooting section below on activating an environment in a job script.

Exporting and Recreating and Conda Environments

Environments can be described by an `environment.yaml` or `requirements.txt` file, which lists packages and optionally their versions. This file can be created from any existing conda environment and used to re-create that environment. Version numbers are required to recreate the environment exactly.

You can create an `environment.yaml` file from an existing environment with the `conda export` command:

```
conda export --name myenv --file=environment.yaml
```

or to create a `requirements.txt` file:

```
conda export --name myenv --file=requirements.txt
```

Given an `environment.yaml` or `requirements.txt` file you can use `conda` or `mamba` to recreate that environment. For example:

```
module load miniforge
conda env create -f environment.yaml
```

or given a `requirements.txt` file:

```
module load miniforge
conda env create --name myenv -f requirements.txt
```

Removing Conda Environments

If your home directory is getting full and you have conda environments you no longer need, you can remove them with the `conda remove` command:

```
module load miniforge
conda remove -n my_env --all
```

Conda/Mamba Environment Location

By default all `conda` and `mamba` environments will be stored in your home directory in `~/.conda`. If you are running out of space in your home directory and you'd like to keep your environments elsewhere, we recommend using your scratch directory. We don't recommend using any disk based storage like your pool or anything under `/orcd/data`.

If you decided to keep conda environments in your scratch directory you will want to "back up" the environment to your home directory by creating a yaml file that will let you recreate the environment.

To change your `conda` and `mamba` install locations create a file called `.condarc` in your home directory. In that file put the following:

.condarc

```
envs_dirs:
  - /home/USERNAME/orcd/scratch/.conda/envs
pkgs_dirs:
  - /home/USERNAME/orcd/scratch/.conda/pkgs
auto_activate_base: false
```

Pros and Cons for Conda Environments

Pros

- Environments can be created with any supported version of Python.
- Can install packages available through various conda channels as well as PyPI (packages installed with `pip`).

- Conda channels include many system libraries, making packages with complicated dependencies easier to install.
- Self-contained environments for each project help stay organized and avoid package dependency conflicts. For software development it allows you to keep better track of your package's dependencies so others know what they need to install.

Cons

- Can not be set to build "on top of" the central installation packages, which means basic package will be re-installed in your home directory.
- Can get very large and take up a lot of space in your home directory.
- It can sometimes be slower than other options.

Troubleshooting Python Package Issues

If you encounter any issues with loading or using Python packages, there are a few steps you can take to troubleshoot.

Check your Python Executable

When you use the `python` command Linux will pick the first `python` executable it finds in your `$PATH`. If Python is not finding your installed packages it is possible that the `python` running is not what you expect. Run the command:

```
which python
```

This will print out the path to the `python` executable that is running. If it doesn't print the right one, check that you've activated your environment or loaded the module you were intending.

Check Python's Path

Python has its own path that gets set when it looks for packages. This path depends on a few things, such as whether you have an environment loaded and how that environment is configured. When in doubt you can view this path in Python with the following commands:

```
import sys
sys.path
```

Similar to the `PATH` environment variable, Python checks the locations on the path in the order they are listed and imports the first of the specified package it finds. If Python is loading the wrong version of a package, checking the path will tell you where to check for the wrong-version package. If Python can't find the package, the path will give you more information about where it is looking.

If in the past you've installed packages with the `--user` flag, Python will pick up those packages before the ones in your environment. If you are using a conda/mamba environment and want Python to *only* use packages in your environment, you can run the following two command:

```
export PYTHONNOUSERSITE=True
```

This will make sure your conda environment packages will be chosen before those that may be installed in your home directory.

Conda/Mamba Init

If you use conda or mamba you may at some point be asked to run `conda init` or `mamba init`. These commands will edit your `.bashrc` file which gets run every time you log in. These additional lines will make permanent changes to your environment that could have unintended effects on your use of the system. This can cause issues, including slowing down your logins and affecting any software builds you try to do.

The best thing to do is to never run these commands, or if you have, to remove the lines that they have added to your `.bashrc` file. Instead load the `miniforge` module before running `conda activate` or `mamba activate`, or you can use the `source activate` command.

If you feel strongly that you want to keep the setup that `conda init` gives you in your `.bashrc`, first be aware that it could cause issues and it might be something to look into removing when troubleshooting other issues. Second, one of the things these lines do is activate the base environment associated with the conda installation, which is usually the source of most of the issues you can run into. You can set conda to not activate the base environment at login by running the following line:

```
conda config --set auto_activate_base false
```

You only need to run this line once. It will edit a configuration file for conda (`.condarc`). You will still be able to run `mamba activate` or `conda activate`.

Environment is not Activating in a Job Script

Getting your python virtual environment or conda/mamba environment to activate in a batch job can sometimes be finicky.

Check your log file to be sure you aren't getting any errors when you try to activate the environment. They may give further instruction on what to do.

Usually the cause is something in your environment that is interfering, these could be:

- The `PYTHONPATH` environment variable is set- this changes where Python looks for packages first. Best practice is to not use `PYTHONPATH` when possible.
- You have another environment activated (see Conda/Mamba Init).
- Your python script starts with a line like `#!/bin/python` or similar. This tells the system to run the script with a python executable that isn't part of your environment. Removing the line will fix the issue.

Beyond this, there are a few things you can try. These won't necessarily fix your environment issues, but might work around them. First let's talk about virtual environments, then conda environments.

Activating a Virtual Environment in a Script

Usually activating the environment in your job script before running your Python script is sufficient.

If your environment isn't activating, specifying the full path to the python executable in your environment sometimes fixes things. Here is a sample job script:

myjob.sh

```
#!/bin/bash

source /path/to/virtual/environment/bin/activate

/path/to/virtual/environment/bin/python myscript.py
```

Be sure to replace `/path/to/virtual/environment` with the actual path to your virtual environment.

Activating a Conda/Mamba Environment in a Script

Usually loading the `miniforge` module and then running `source activate myenv` will work to activate a conda or mamba environment in a job script (as shown above).

If this doesn't work sometimes adding `eval "$(conda shell.bash hook)"` in the line before activating the environment will make it work:

mycondajob.sh

```
#!/bin/bash

module load miniforge
eval "$(conda shell.bash hook)"

conda activate myenv # or `mamba activate myenv` for mamba environments

python myscript.sh
```

December 16, 2025